Lesson 5.7 **Duration: 40 minutes**

Sorting Algorithms

Learning Objectives

- Implement bubble sort and insertion sort algorithms
- Compare the time complexity of different sorting approaches
- Select appropriate sorting algorithms based on data characteristics

Engage (5-7 minutes)

Give students a deck of playing cards and ask them to sort by number. Observe different strategies: some insert cards one by one (insertion sort), others compare adjacent pairs and swap (bubble sort). Discuss which strategy is faster and why—it depends on the initial arrangement of data.

Explore (10-12 minutes)

In groups, students physically sort arrays of numbered cards using bubble sort and insertion sort. They record every comparison and swap on a worksheet. After completing both, they compare the number of operations for different initial orderings (already sorted, reverse sorted, random). Groups graph their results to visualize performance differences.

Explain (10-12 minutes)

Sorting algorithms arrange data in ascending or descending order. Bubble sort repeatedly compares adjacent elements and swaps them if out of order, "bubbling" largest elements to the end—$O(n^2)$ worst case. Insertion sort builds a sorted portion by inserting each element into its correct position—also $O(n^2)$ but efficient for nearly sorted data. More advanced algorithms like merge sort ($O(n \log n)$) use divide-and-conquer. The choice depends on data size, initial order, and memory constraints.

Elaborate (8-10 minutes)

Trace bubble sort and insertion sort with arrays of different sizes. Analyze best-case, worst-case, and average-case performance. Discuss space complexity—in-place algorithms vs. those requiring extra memory. Examine why merge sort achieves $O(n \log n)$ by splitting and merging, and introduce quicksort's partitioning strategy conceptually.

Evaluate (5-8 minutes)

Students implement both bubble sort and insertion sort in pseudocode, then perform a complexity analysis for each. Assessment: correctness of algorithm, accurate operation counting, and justified

algorithm selection for given scenarios (e.g., small dataset vs. large dataset, nearly sorted vs. random).